ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО	УТВЕРЖДЕНО
Приглашенный преподаватель	Академический руководитель образовательной программы «Программная инженерия», старший преподаватель департамента программной инженерии
_____ С. Б. Гурин	_____ Н. А. Павлочев
«___» _____ 2026 г.	«___» _____ 2026 г.

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

Sentinel — Интеллектуальный тайм-менеджер

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.06.05-01 51 01-1-ЛУ

Исполнитель:
Студент группы БПИ236
_____ / Т. А. Петянов /
«___» _____ 2026 г.

Москва 2026

УТВЕРЖДЕН

RU.17701729.06.05-01 51 01-1-ЛУ

Sentinel — Интеллектуальный тайм-менеджер

Программа и методика испытаний

RU.17701729.06.05-01 51 01-1

Листов 28

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

Москва 2026

АННОТАЦИЯ

Настоящий программный документ представляет собой программу и методику испытаний к программному проекту «Sentinel — Интеллектуальный тайм-менеджер».

В разделе «Объект испытаний» указано наименование, краткая характеристика и назначение программы.

В разделе «Цель испытаний» указана цель проведения испытаний.

Раздел «Требования к программе» содержит основные требования к программе, которые подлежат проверке во время испытаний (требования к функционалу и интерфейсу).

Раздел «Требования к программным документам» содержит состав программной документации, которая представляется на испытания.

Раздел «Средства и порядок испытаний» содержит информацию о технических и программных средствах, которые следует использовать во время испытаний, а также порядок этих испытаний.

Раздел «Методы испытаний» содержит информацию об используемых методах испытаний

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 Виды программ и программных документов[1].
2. ГОСТ 19.102-77 Стадии разработки[2].
3. ГОСТ 19.103-77 Обозначения программ и программных документов[3].
4. ГОСТ 19.104-78 Основные надписи[4].
5. ГОСТ 19.105-78 Общие требования к программным документам[5].
6. ГОСТ 19.106-78 Требования к программным документам, выполненным печатным способом[6].
7. ГОСТ 19.301-79 Программа и методика испытаний. Требования к содержанию и оформлению[7].

Изменения к данной Программе и методике испытаний оформляются согласно ГОСТ 19.603-78[8], ГОСТ 19.604-78[9].

СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ 4
1.1. Наименование программы 4
1.2. Краткая характеристика области применения программы 4
2. ЦЕЛЬ ИСПЫТАНИЙ 5
3. ТРЕБОВАНИЯ К ПРОГРАММЕ 6
3.1. Требования к функциональным характеристикам 6
3.1.1. Требования к составу выполняемых функций 6
3.1.2. Требования к организации входных данных 10
3.1.3. Требования к организации выходных данных 11
3.1.4. Требования к временным характеристикам 12
3.1.5. Требования к интерфейсу 12
3.2. Требования к надежности 13
3.2.1. Требования к обеспечению надежного функционирования программ 13
3.2.2. Время восстановления после отказа 13
3.2.3. Отказы из-за некорректных действий оператора 14
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ 15
4.1. Состав программной документации 15
4.2. Специальные требования к программной документации 15
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ 16
5.1. Состав технических и программных средств 16
5.2. Порядок проведения испытаний 16
6. МЕТОДЫ ИСПЫТАНИЙ 18
6.1. Проверка требований к программной документации 18
6.2. Интеграционное тестирование 18
6.3. Тестирование функциональности 21
6.4. Проверка требований к надежности 23
ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ 25
ПРИЛОЖЕНИЕ 2. ТЕРМИНЫ И СОКРАЩЕНИЯ 27

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование программы

Наименование программы: «Sentinel — Интеллектуальный тайм-менеджер».

Наименование программы на английском языке: «Sentinel — AI Scheduler».

Краткое наименование программы для пользователя: «Sentinel».

1.2. Краткая характеристика области применения программы

Разрабатываемое приложение представляет собой мобильную iOS-программу для интеллектуального планирования личного расписания. Основной сценарий работы приложения состоит в преобразовании свободного текстового запроса пользователя, а также прикреплённых изображений, в набор структурированных календарных событий. Полученные предложения пользователь просматривает, при необходимости редактирует через выбор предложений ассистента и затем сохраняет в календарь устройства.

Система учитывает индивидуальные ограничения и контекст пользователя через модель «Resource Battery», реализованную с использованием Foundation Models, поддерживает автоматическую балансировку порядка событий «Rebalance», ведёт историю диалогов и хранит пользовательские настройки. Для взаимодействия с календарём устройства используется EventKit, для хранения локальных записей синхронизации — SwiftData, для хранения пользовательской сессии — Keychain, для базовых настроек — UserDefaults.

Приложение может применяться в следующих сценариях:

1. Планирование учебных занятий, дедлайнов и личных дел на основе свободного текстового описания.

2. Преобразование одного пользовательского запроса в несколько календарных событий с последующим выбором нужных предложений.

3. Обработка фотографий и файлов изображений для извлечения задач, дат и временных ограничений.

4. Анализ занятости пользователя на ближайший день или на выбранный период с помощью «Resource Battery».

5. Ручное управление календарными событиями и синхронизация с календарём устройства.

6. Предварительный просмотр и применение сценария автоматической ребалансировки расписания.

7. Отслеживание пользовательского прогресса через систему достижений.

2. ЦЕЛЬ ИСПЫТАНИЙ

Целью испытаний является проверка корректности выполнения программой функций, изложенных в разделе «Требования к программе» документа «Техническое задание», а также подтверждение того, что реализованное приложение, сопровождающая документация и автоматизированные проверки образуют согласованный комплект, пригодный для приемки и дальнейшего сопровождения.

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

3.1. Требования к функциональным характеристикам

3.1.1. Требования к составу выполняемых функций

iOS-приложение должно реализовывать следующие функции:

1. Главный экран «Сегодня»

1.1. Сценарий неавторизованного пользователя

1.1.1. Отображение информационных карточек приложения.

1.1.2. Кнопка входа в аккаунт.

1.1.3. Кнопка создания аккаунта.

1.2. Сценарий авторизованного пользователя

1.2.1. Отображение текущей даты и приветственного заголовка.

1.2.2. Кнопка перехода к экрану профиля.

1.2.3. Карточка расписания на текущий день с количеством событий и краткой информацией о ближайшем событии.

1.2.4. Карточка «Resource Battery» с крупным значением, иконкой состояния и поясняющим текстом.

1.2.5. Индикация состояний «Resource Battery»: hidden, placeholder, setupRequired(.downloadModel), setupRequired(.enableAppleIntelligence), ready.

1.2.6. Кнопка перехода к полному календарю.

1.2.7. Кнопка запуска сценария ребалансировки.

1.2.8. Переход к полному списку достижений.

2. Экран «Чат»

2.1. Работа с диалогом

2.1.1. Отправка текстовых сообщений.

2.1.2. Автоматическое открытие последнего активного чата, если с момента его последнего показа прошло не более 10 минут.

2.1.3. Автоматическое создание нового чата, если правило 10 минут не выполнено.

2.1.4. Загрузка списка чатов и выбор активного диалога.

2.1.5. Загрузка более ранних сообщений.

2.1.6. Отображение ошибок загрузки истории и действие «Повторить».

2.1.7. Сохранение неотправленного сообщения в ленте с индикатором ошибки.

2.1.8. Действия Retry и Remove для сообщения, отправка которого завершилась ошибкой.

2.2. Работа с вложениями

2.2.1. Кнопка добавления вложения.

2.2.2. Открытие модальной панели выбора вложений.

2.2.3. Кнопка All Photos для перехода к полному выбору изображений.

2.2.4. Горизонтальный список последних изображений из фотогалереи.

2.2.5. Кнопка камеры.

2.2.6. Строка Add files для открытия системного файлового пикера.

2.2.7. Однократное согласие на серверную обработку вложений перед первой отправкой изображений или файлов.

2.2.8. Прикрепление фотографий форматов PNG, JPG, JPEG и HEIC из галереи или камеры.

2.2.9. Прикрепление файлов изображений поддерживаемых форматов через системный пикер.

2.2.10. Удаление прикрепленного изображения или файла до отправки сообщения.

2.2.11. Сохранение неотправленного вложения вместе с сообщением для повторной отправки.

2.2.12. Проверка размера и количества вложений до начала отправки.

2.3. Работа с предложениями событий

2.3.1. Отображение блока предложенных событий в сообщении ассистента.

2.3.2. Разворачивание и сворачивание блока предложений.

2.3.3. Выбор одного или нескольких предложений событий.

2.3.4. Кнопка подтверждения выбранных предложений.

2.3.5. Отображение конфликтов с текущим расписанием.

2.3.6. Синхронизация состояния предложений после применения.

3. Экран списка чатов

3.1. Просмотр сохранённых чатов.

3.2. Выбор нужного чата для продолжения диалога.

3.3. Кнопка создания нового чата.

3.4. Кнопка возврата к активному диалогу.

3.5. Удаление чата через жестовое действие или соответствующий UI-контрол.

4. Экран календаря

4.1. Просмотр расписания

4.1.1. Представление недели и месяца.

4.1.2. Открытие и закрытие графического выбора месяца.

4.1.3. Выбор дня в недельной полосе.

4.1.4. Отображение секций по дням.

4.1.5. Отображение day-level индикатора «Resource Battery» для каждой секции дня.

4.2. Управление событиями

4.2.1. Кнопка создания нового события.

4.2.2. Открытие редактора события по нажатию на строку события.

4.2.3. Действия редактирования и удаления события.

4.2.4. Обновление календарных данных после сохранения или удаления события.

5. Экран редактора события

5.1. Поле заголовка события.

5.2. Поле местоположения.

5.3. Поле заметок.

5.4. Переключатель типа сущности: событие или напоминание.

5.5. Переключатель all-day.

5.6. Переключатель fixed event.

5.7. Поля выбора даты и времени начала и окончания.

5.8. Кнопка закрытия редактора.

5.9. Кнопка сохранения события.

6. Экран профиля

6.1. Отображение имени и почты пользователя.

6.2. Debug-only блок Environment, расположенный под профилем и над блоком Default Prompt.

6.3. Поле редактирования Default Prompt.

6.4. Автоматическое сохранение Default Prompt при потере фокуса, закрытии экрана и завершении редактирования.

6.5. Автоматическое добавление Default Prompt к каждому сообщению в чате и к каждому запросу ребалансировки.

6.6. Строка перехода к списку юридических документов.

6.7. Переход на экран достижений.

6.8. Кнопка выхода из учётной записи.

6.9. Кнопка удаления учётной записи.

6.10. Поле ввода текущего пароля при удалении учётной записи.

6.11. Кнопка подтверждения удаления учётной записи.

6.12. Кнопка отмены удаления учётной записи.

7. Экран достижений

7.1. Отображение сводной карточки прогресса.

7.2. Отображение групп достижений и уровней внутри группы.

7.3. Кнопка повторной загрузки данных при ошибке.

7.4. Возможность поделиться разблокированным достижением через системный ShareLink.

8. Экран списка юридических документов

8.1. Отображение списка юридических документов приложения.

8.2. Строки Privacy Policy, Terms of Use, Consent to Personal Data Processing, Attachment Processing Notice и иных обязательных документов.

8.3. Открытие выбранного документа поверх приложения во встроенном Safari controller.

9. Debug-only элемент Environment

9.1. Элемент доступен только в тестовых сборках приложения.

9.2. Элемент расположен на экране Profile под блоком профиля и над блоком Default Prompt.

9.3. Выбор текущего окружения выполняется через встроенное меню со значениями Local, Production и Testing.

10. Экран ребалансировки

10.1. Выбор дней для ребалансировки.

10.2. Отображение количества событий по каждому выбранному дню.

10.3. Отображение day-level индикатора «Resource Battery» для выбранных дней.

10.4. Кнопка предварительного просмотра ребалансировки.

10.5. Экран предварительного просмотра изменений расписания.

10.6. Кнопка обновления предварительного просмотра.

10.7. Кнопка применения изменений.

10.8. Кнопка закрытия экрана ребалансировки.

11. Экран авторизации

11.0. Общие элементы

11.0.1. Кнопка закрытия экрана авторизации.

11.0.2. Отображение ссылок на Privacy Policy и Terms of Use в нижней части экрана.

11.1. Экран входа

11.1.1. Поле ввода email.

11.1.2. Поле ввода пароля.

11.1.3. Кнопка входа.

11.1.4. Кнопка перехода к регистрации.

11.1.5. Кнопка перехода к восстановлению пароля.

11.2. Экран регистрации, шаг 1

11.2.1. Поле ввода email.

11.2.2. Кнопка продолжения.

11.2.3. Кнопка возврата ко входу.

11.2.4. Отображение текста согласия с пользовательскими документами.

11.3. Экран регистрации, шаг 2

11.3.1. Поле ввода пароля.

11.3.2. Поле подтверждения пароля.

11.3.3. Кнопка завершения регистрации.

11.3.4. Кнопка возврата к шагу ввода email.

11.4. Экран подтверждения почты

11.4.1. Поле ввода verification token.

11.4.2. Кнопка подтверждения почты.

11.4.3. Кнопка повторной отправки письма подтверждения.

11.4.4. Кнопка повторной попытки входа после подтверждения.

11.4.5. Кнопка возврата ко входу.

11.5. Экран восстановления пароля

11.5.1. Поле ввода email для отправки письма восстановления.

11.5.2. Кнопка отправки ссылки на восстановление.

11.5.3. Поле ввода reset token.

11.5.4. Поле ввода нового пароля.

11.5.5. Поле подтверждения нового пароля.

11.5.6. Кнопка завершения сброса пароля.

11.5.7. Кнопка возврата ко входу.

3.1.2. Требования к организации входных данных

Входные данные представляют собой:

1. Сообщения в чате

1.1. Текст до 2000 символов в кодировке UTF-8.

1.2. Фотографии форматов PNG, JPG, JPEG и HEIC; до 10 МБ на файл; до 10 изображений на сообщение.

1.3. Файлы изображений поддерживаемых форматов через системный пикер.

2. Данные авторизации и управления учётной записью

2.1. Адрес электронной почты.

2.2. Пароль пользователя.

2.3. Verification token.

2.4. Reset token.

3. Данные календарных сущностей

3.1. Заголовок события.

3.2. Местоположение.

3.3. Описание.

3.4. Дата и время начала.

3.5. Дата и время окончания.

3.6. Признаки all-day, fixed, reminder или event.

4. Данные ребалансировки и пользовательского контекста

4.1. Выбранные для ребалансировки дни.

4.2. Значение Default Prompt.

4.3. Текущая временная зона устройства.

4.4. Выбранное окружение приложения в тестовой сборке.

5. Правила приёма и валидации

5.1. Проверка длины сообщения и допустимых символов.

5.2. Проверка типа, размера и количества вложений.

5.3. Проверка непустого заголовка события перед сохранением.

5.4. Проверка обязательных полей сценариев регистрации, подтверждения почты и восстановления пароля.

5.5. Запрос однократного согласия на серверную обработку вложений перед первой отправкой файлов или изображений.

3.1.3. Требования к организации выходных данных

1. Внутренние сущности

1.1. Пользователь — владелец учётной записи и персональных настроек.

1.2. Чат — отдельный диалог пользователя с ассистентом.

1.3. Сообщение чата — отдельная запись диалога с текстом, изображениями и, при наличии, структурированными предложениями событий.

1.4. Событие — запись расписания с идентификатором, временем, параметрами и происхождением.

1.5. Достижение — запись о прогрессе пользователя, уровне и статусе разблокировки.

2. Внешние артефакты

2.1. Ответы серверной части в формате JSON по контрактам REST API.

2.2. События календаря устройства, синхронизированные через EventKit.

2.3. Юридические документы, отображаемые внутри приложения.

2.4. Служебные сообщения о статусе операций и ошибках.

3. Локально сохраняемые данные

3.1. Записи синхронизации календаря в SwiftData.

3.2. Пользовательская сессия в Keychain.

3.3. AppSettings и service-флаги в UserDefaults.

4. Требования к целостности выходных данных

4.1. Исключение дублирования событий при экспорте в календарь устройства.

4.2. Сохранение признака конфликта по времени для предложенных и существующих событий.

4.3. Сохранение связи между серверными идентификаторами событий и локальными событиями календаря.

3.1.4. Требования к временным характеристикам

1. Реакции интерфейса, не требующие сетевого запроса, должны выполняться без визуально заметной задержки для пользователя.

2. Переключение экранов и отображение листов не должно сопровождаться зависанием интерфейса.

3. Целевое время интерпретации пользовательского запроса на серверной стороне составляет 3–6 секунд; предельное время ожидания ответа — 15 секунд.

4. Операция создания или обновления календарного события через UI приложения должна выполняться в пределах нормального пользовательского ожидания без блокировки интерфейса.

5. Автосохранение Default Prompt должно происходить при завершении редактирования без дополнительной кнопки сохранения.

3.1.5. Требования к интерфейсу

1. Общие требования

1.1. Соответствие Apple Human Interface Guidelines.

1.2. Поддержка светлой и тёмной темы оформления.

1.3. Поддержка Dynamic Type.

1.4. Минимальный размер интерактивных элементов — 44×44 pt.

1.5. Поддержка VoiceOver и корректных accessibility-меток.

1.6. Контраст визуальных элементов не ниже требований WCAG 2.1 AA.

2. Организация экранов

2.1. Основной пользовательский сценарий «чат → предложения событий → подтверждение → календарь» должен быть доступен без перехода в служебные разделы.

2.2. Пользовательские настройки должны быть сосредоточены на экране Profile.

2.3. Юридические документы должны открываться модально поверх приложения.

2.4. Environment должен быть доступен только в тестовых сборках.

3. Индикация состояний

3.1. Отображение статуса загрузки для сетевых операций.

3.2. Отображение текстовых ошибок и действий повторной попытки.

3.3. Отображение состояний «Resource Battery».

3.4. Отображение ошибок неуспешной отправки сообщений с возможностью Retry или Remove.

3.2. Требования к надежности

3.2.1. Требования к обеспечению надежного функционирования программ

1. Устойчивость работы приложения

1.1. Валидация входных данных перед запуском критических операций.

1.2. Контролируемая обработка сетевых ошибок, ошибок API и локальных отказов без нарушения целостности пользовательского состояния.

1.3. Идемпотентность операций синхронизации календаря для предотвращения дублирования событий.

1.4. Сохранение секретов и токенов в защищённом хранилище Keychain.

1.5. Сохранение локальных настроек пользователя вне volatile-состояния UI.

1.6. Сохранение видимости конфликтов расписания для пользователя.

3.2.2. Время восстановления после отказа

1. Восстановление пользовательского контекста

1.1. Автоматическое восстановление пользовательской сессии при наличии сохранённого токена.

1.2. Восстановление Default Prompt и выбранного окружения приложения.

1.3. Восстановление идентификатора недавно активного чата в пределах окна актуальности.

1.4. Повторная загрузка календарных событий и серверных данных после перезапуска приложения.

3.2.3. Отказы из-за некорректных действий оператора

1. Некорректные действия пользователя в auth-сценариях

1.1. Проверка обязательности email и пароля при входе.

1.2. Проверка обязательности verification token при подтверждении почты.

1.3. Проверка обязательности reset token и новых учётных данных при восстановлении пароля.

2. Некорректные действия пользователя при работе с сообщениями и вложениями

2.1. Проверка ограничения размера одного вложения.

2.2. Проверка ограничения общего количества вложений в сообщении.

2.3. Проверка сохранения ошибочно неотправленного сообщения в управляемом состоянии с возможностью Retry или Remove.

3. Некорректные действия пользователя при работе с календарём и профилем

3.1. Проверка обязательности непустого заголовка события перед сохранением.

3.2. Проверка обязательности текущего пароля при удалении учётной записи.

3.3. Проверка корректного поведения при закрытии экрана с редактируемым Default Prompt.

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

4.1. Состав программной документации

1. «Sentinel — Интеллектуальный тайм-менеджер». Техническое задание (ГОСТ 19.201-78).
2. «Sentinel — Интеллектуальный тайм-менеджер». Программа и методика испытаний (ГОСТ 19.301-79).
3. «Sentinel — Интеллектуальный тайм-менеджер». Пояснительная записка (ГОСТ 19.404-79).
4. «Sentinel — Интеллектуальный тайм-менеджер». Руководство оператора (ГОСТ 19.505-79).
5. «Sentinel — Интеллектуальный тайм-менеджер». Текст программы (ГОСТ 19.401-78).

4.2. Специальные требования к программной документации

Документы к программе должны быть выполнены в соответствии с ГОСТ 19.106-78 и ГОСТами к каждому виду документа. Тексты документов должны быть согласованы между собой по наименованию программы, составу функций, архитектурным решениям, ограничениям платформы и описанию пользовательских сценариев.

5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

5.1. Состав технических и программных средств

Для проведения испытаний использовались следующие технические и программные средства:

1. Технические средства
   1.1. Компьютер Apple Mac с установленной средой разработки Xcode.
   1.2. Симулятор iPhone 17 Pro с версией iOS 26.2.
   1.3. Файловая система разработки с локальной копией репозитория проекта Sentinel.
2. Программные средства
   2.1. Xcode 26.2.
   2.2. Схема проекта «Sentinel».
   2.3. Средства запуска и анализа автоматизированных тестов Xcode.
   2.4. Фреймворк Swift Testing.
   2.5. TCA TestStore для проверки переходов состояний.
   2.6. Средство анализа покрытия кода xccov.
3. Испытуемые артефакты
   3.1. Исходный код приложения Sentinel, включая основной таргет приложения и локальный Swift package SentinelModules с библиотекой SentinelUI.
   3.2. Документы «Техническое задание», «Пояснительная записка», «Руководство оператора» и «Программа и методика испытаний».
   3.3. Автоматизированный тестовый набор таргета SentinelTests.

5.2. Порядок проведения испытаний

Испытания проводились в следующем порядке:

1. Анализ действующего комплекта проектной документации и проверка согласованности терминологии.
2. Сопоставление требований документа «Техническое задание» с реальными представлениями, редьюсерами, клиентами и тестами проекта.
3. Запуск автоматизированного тестового набора проекта на симуляторе iPhone 17 Pro с iOS 26.2 через Xcode 26.2.
4. Анализ состава пройденных тестов и итогового покрытия кода.

5. Сопоставление результатов автоматизированных проверок с основными пользовательскими сценариями Sentinel.
6. Фиксация результатов испытаний, выявленных ограничений и областей дальнейшего наращивания тестовой базы.

6. МЕТОДЫ ИСПЫТАНИЙ

6.1. Проверка требований к программной документации

Проверка требований к программной документации выполнялась путём анализа состава документов, согласованности их структуры и соответствия их содержания текущему продукту Sentinel.

1. Проверка состава документации

1.1. Проверено наличие технического задания.

1.2. Проверено наличие пояснительной записки.

1.3. Проверено наличие руководства оператора.

1.4. Проверено наличие текста программы.

1.5. Проверено наличие программы и методики испытаний.

2. Проверка согласованности наименований и терминологии

2.1. Проверено единое использование наименования «Sentinel — Интеллектуальный тайм-менеджер».

2.2. Проверено единое использование английского наименования «Sentinel — AI Scheduler».

2.3. Проверено единое использование краткого наименования «Sentinel».

2.4. Проверено единое понимание терминов «Profile», «Default Prompt», «Resource Battery», «Rebalance», «Environment».

3. Проверка содержательной согласованности

3.1. Проверено, что пользовательские настройки сосредоточены на экране Profile.

3.2. Проверено, что legal-flow описан через список документов и модальное открытие документа внутри приложения.

3.3. Проверено, что Environment описан как debug-only функция тестовой сборки.

3.4. Проверено, что Default Prompt описан как автоматически сохраняемый и автоматически добавляемый к сообщениям и запросам ребалансировки.

4. Результаты проверки

4.1. Комплект основных документов по проекту присутствует.

4.2. Наименования программы и основные продуктовые термины приведены к согласованному виду.

4.3. Настоящий документ синхронизирован с техническим заданием, пояснительной запиской, руководством оператора и фактическим кодом приложения.

6.2. Интеграционное тестирование

Автоматизированные испытания проводились штатными средствами Xcode по схеме «Sentinel» на симуляторе iPhone 17 Pro с iOS 26.2. Для написания проверок используется Swift Testing. Для тестирования редьюсеров The Composable Architecture используется TestStore, позволяющий проверять начальное состояние, последовательность действий, асинхронные эффекты и итоговое состояние модуля.

1. Тесты сетевых контрактов и транспортного слоя

1.1. Файл «APIRequestAndErrorTests.swift» проверяет построение HTTP-запросов, формирование заголовков, query-параметров и декодирование ошибок API.

1.2. Файл «DTOCodingTests.swift» проверяет кодирование и декодирование DTO авторизации, событий, чатов, сообщений, достижений и структурированного контента.

1.3. Файл «APIModelConverterTests.swift» проверяет преобразование транспортных DTO в доменные сущности приложения, включая обработку fallback-ветвей.

2. Тесты сессии и настроек приложения

2.1. Файл «SessionModelTests.swift» проверяет корректность сериализации и десериализации аутентифицированной пользовательской сессии.

2.2. Файл «AppConfigurationAndSettingsTests.swift» проверяет адреса окружений, юридические документы, поведение AppSettings, сохранение и повторную загрузку пользовательских настроек.

2.3. Файл «DefaultPromptEnvelopeTests.swift» проверяет добавление и удаление обёртки Default Prompt в содержимом сообщений.

3. Тесты доменных моделей и производных состояний

3.1. Файл «HomeStateDerivedTests.swift» проверяет вычисляемые свойства домашнего экрана, карточки расписания и «Resource Battery».

3.2. Файл «CalendarStateDerivedTests.swift» проверяет построение payload редактора события, формирование секций повестки дня, вычисление конфликтов и day-level battery request.

3.3. Файл «AchievementAndAuthStateTests.swift» проверяет производные свойства auth-сценария и модели достижений.

3.4. Файл «ChatDomainModelTests.swift» проверяет доменные модели чата, предложений событий, сообщений и вложений.

3.5. Файл «ChatThreadSupportTests.swift» проверяет вспомогательную логику формирования заголовков чатов, объединения сообщений и обновления предложений.

3.6. Файлы «AdditionalPureCoverageTests.swift» и «BranchCoverageTests.swift» закрывают дополнительные чистые ветви и вспомогательные сценарии.

4. Тесты auth-flow

4.1. Файл «AuthReducerFlowTests.swift» проверяет переходы между режимами login, register, verification pending и forgot password.

4.2. Проверяются ветви успешного входа, регистрации, подтверждения почты, повторной отправки письма подтверждения, отправки письма восстановления и завершения сброса пароля.

4.3. Проверяются обязательность email, пароля, verification token и reset token.

5. Тесты чатового сценария

5.1. Файл «ReducerFlowTests.swift» проверяет загрузку списка чатов, выбор активного чата, удаление чатов, начало сценария отправки сообщения и применение предложений событий.

5.2. Файл «ChatThreadFeatureFlowTests.swift» проверяет сброс состояния при смене токена или чата, загрузку истории, валидацию вложений, работу блока предложений и ветви ошибки отправки сообщения.

5.3. Файл «ChatAttachmentPickerFeatureTests.swift» проверяет одноразовый consent, открытие панели выбора вложений и добавление recent photo в compose-состояние.

6. Тесты календаря и ребалансировки

6.1. Файл «CalendarReducerFlowTests.swift» проверяет создание, редактирование и удаление событий, перезагрузку событий по дате, сохранение ошибок и работу day-level «Resource Battery».

6.2. Файл «RebalanceFeatureFlowTests.swift» проверяет загрузку доступных дней, подгрузку Default Prompt, выбор дней, получение preview и применение ребалансировки.

7. Тесты навигации и визуального слоя

7.1. Файл «HomeProfileNavigationFeatureTests.swift» проверяет переходы между Home, Calendar, Achievements и Profile.

7.2. Файл «ViewSmokeTests.swift» проверяет создание и базовый рендер ключевых экранов и состояний: Home, Calendar, Profile, Achievements, Chat UI и Rebalance.

8. Результаты автоматизированных испытаний

8.1. Успешно выполнено 82 теста, сгруппированных в 21 suite.

8.2. Все тесты завершились со статусом «TEST SUCCEEDED».

8.3. В интерфейсе Xcode общее покрытие клиентского приложения отображается как 58%. Детализированный отчёт xccov для этого же результата испытаний составляет 58,28%, что соответствует 8482 покрытым строкам из 14555 исполняемых строк.

8.4. Критические участки чатового сценария покрыты существенно выше среднего: «ChatThreadSupport+Apply.swift» — 96,72%, «ChatThreadSupport+Send.swift» — 92,53%.

8.5. Вспомогательная логика списка чатов и типов ребалансировки также покрыта выше среднего: «ChatListFeature+Support.swift» — 94,44%, «RebalanceFeature+Types.swift» — 100,00%.

8.6. Значение общего покрытия ниже покрытия критических редьюсеров из-за того, что значительная часть строк приложения приходится на SwiftUI-представления и визуальные ветви, которые в текущем комплекте проверяются преимущественно smoke-тестами, а не полноценным UI automation.

6.3. Тестирование функциональности

Функциональные испытания выполнялись путём сопоставления автоматизированных flow-тестов, smoke-проверок визуального слоя и фактической структуры представлений и редьюсеров проекта с основными пользовательскими сценариями Sentinel.

1. Сценарий авторизации

1.1. Проверялись вход по email и паролю, регистрация в два шага, подтверждение почты, повторная отправка письма подтверждения и восстановление пароля.

1.2. Проверялись обязательность полей и корректный сброс временных полей при переключении между режимами auth-flow.

1.3. Результат: сценарий авторизации подтверждён тестами «AuthReducerFlowTests.swift» и производными состояниями «AchievementAndAuthStateTests.swift».

2. Сценарий работы со списком чатов и активным диалогом

2.1. Проверялись загрузка списка чатов, выбор первого активного чата, удаление чата, переход к новому чату и обновление активного идентификатора.

2.2. Проверялась загрузка истории сообщений, включая подгрузку более ранних сообщений и ветви ошибки.

2.3. Результат: сценарий подтверждён тестами «ReducerFlowTests.swift» и «ChatThreadFeatureFlowTests.swift».

3. Сценарий отправки сообщения и вложений

3.1. Проверялись отправка текстового сообщения, автоматическое создание чата при первой отправке, подготовка и отправка вложения.

3.2. Проверялись однократное согласие на серверную обработку вложений, загрузка recent photos, добавление изображений в compose-состояние, а также ограничения на размер и количество вложений.

3.3. Проверялись ветви неуспешной отправки сообщения, при которых сообщение остаётся в ленте и переходит в состояние failed.

3.4. Результат: сценарий подтверждён тестами «ChatThreadFeatureFlowTests.swift», «ChatAttachmentPickerFeatureTests.swift», «ReducerFlowTests.swift» и доменными моделями «ChatDomainModelTests.swift».

4. Сценарий применения предложенных событий

4.1. Проверялись формирование блока предложений событий, выбор предложений, вычисление конфликтов и построение плана применения.

4.2. Проверялись отправка на сервер информации о выбранных действиях, обновление сообщения ассистента после применения и последующая синхронизация календарных данных.

4.3. Результат: критический сценарий «чат → предложения событий → подтверждение → календарь» подтверждён тестами «ReducerFlowTests.swift», «ChatThreadFeatureFlowTests.swift» и «ChatThreadSupportTests.swift».

5. Сценарий календаря и ручного редактирования событий

5.1. Проверялись загрузка событий по диапазону дат, переключение между днями и месяцами, открытие редактора события, сохранение и удаление записи.

5.2. Проверялись вычисление строк повестки дня, признаки конфликтов, отображение reminder/event и построение day-level battery request.

5.3. Результат: сценарий подтверждён тестами «CalendarReducerFlowTests.swift» и «CalendarStateDerivedTests.swift».

6. Сценарий Profile, legal-flow и настроек пользователя

6.1. Проверялись загрузка пользовательских настроек, редактирование Default Prompt, автосохранение Default Prompt и изменение выбранного окружения.

6.2. Проверялись наличие legal-flow, состава юридических документов и корректность URL для открываемых документов.

6.3. Проверялись выход из учётной записи и удаление учётной записи с обязательным подтверждением паролем.

6.4. Результат: сценарий подтверждён тестами «AppConfigurationAndSettingsTests.swift», «DefaultPromptEnvelopeTests.swift», «HomeProfileNavigationFeatureTests.swift» и анализом редьюсера ProfileFeature.

7. Сценарий достижений и домашнего экрана

7.1. Проверялись построение карточек домашнего экрана, вычисляемые свойства расписания и «Resource Battery», а также агрегированные свойства достижений.

7.2. Проверялись переходы с Home к календарю и достижениям, а из Profile — к достижениям.

7.3. Результат: сценарий подтверждён тестами «HomeStateDerivedTests.swift», «AchievementAndAuthStateTests.swift» и «HomeProfileNavigationFeatureTests.swift».

8. Сценарий Rebalance

8.1. Проверялись загрузка доступных дней, автоматическая подгрузка Default Prompt, выбор дней, preview, refresh и apply.

8.2. Проверялась синхронизация календарных данных после применения ребалансировки и обновление соответствующих состояний.

8.3. Результат: сценарий подтверждён тестами «RebalanceFeatureFlowTests.swift».

9. Ограничения функциональных испытаний

9.1. Отдельный набор XCUITest-сценариев на момент проведения испытаний отсутствует.

9.2. Полноценные end-to-end проверки камеры, системных разрешений и реального календаря устройства не включены в текущий автоматизированный набор и должны наращиваться в следующих итерациях.

6.4. Проверка требований к надежности

Проверка надежности выполнялась на основе автоматизированных тестов редьюсеров, доменных моделей и настроек приложения, анализирующих ветви валидации, error-handling и восстановления состояния.

1. Проверка устойчивости к некорректному вводу

1.1. Проверялась обязательность email и пароля в auth-сценариях.

1.2. Проверялась обязательность verification token и reset token.

1.3. Проверялась обязательность заголовка события при сохранении.

1.4. Проверялась реакция на слишком большой файл и превышение лимита вложений.

1.5. Результат: все перечисленные ветви обрабатываются контролируемо и завершаются сообщением пользователю без аварийного завершения модуля.

2. Проверка поведения при ошибках операций

2.1. Проверялась реакция на неуспешную отправку сообщения.

2.2. Проверялась реакция на ошибку удаления события.

2.3. Проверялась реакция на ошибку сохранения события.

2.4. Проверялась ветвь login с ответом verification required.

2.5. Результат: ошибки переводят модули в предсказуемое состояние, сохраняют необходимые пользовательские данные и формируют читаемое сообщение об ошибке.

3. Проверка восстановления и сброса состояния

3.1. Проверялась попытка восстановления сессии при отсутствии сохранённых данных.

3.2. Проверялся полный сброс чувствительного состояния при потере access token.

3.3. Проверялось сохранение и повторная загрузка AppSettings, включая Default Prompt, selectedEnvironment и сведения о недавно активном чате.

3.4. Результат: логика восстановления и сброса состояния подтверждена тестами «AuthReducerFlowTests.swift», «AppConfigurationAndSettingsTests.swift» и «ChatThreadFeatureFlowTests.swift».

4. Проверка целостности локальных и серверных данных

4.1. Проверялись сериализация и десериализация пользовательской сессии.

4.2. Проверялись транспортные контракты событий, чатов, сообщений и достижений.

4.3. Проверялись построение плана применения выбранных предложений событий и расчёт диапазона повторной синхронизации.

4.4. Результат: ключевые механизмы целостности подтверждены тестами «SessionModelTests.swift», «DTOCodingTests.swift», «APIModelConverterTests.swift» и «ChatDomainModelTests.swift».

5. Общий вывод по надежности

5.1. Проверенные сценарии подтверждают устойчивость критических частей доменной логики к типовым ошибкам ввода и отказам внешних операций.

5.2. Ограничением текущего этапа остаётся отсутствие отдельного набора длительных нагрузочных испытаний и полного UI automation.

5.3. Указанное ограничение не отменяет результата выполненного модульного и flow-тестирования, но должно учитываться при дальнейшем развитии тестовой базы проекта.

ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. Apple Developer Documentation. EventKit. // Электронный ресурс. URL: https://developer.apple.com/documentation/eventkit (дата обращения 29.04.26).
15. Apple Developer Documentation. SwiftUI. // Электронный ресурс. URL: https://developer.apple.com/documentation/swiftui (дата обращения 29.04.26).

16. Apple Developer Documentation. UserDefaults. // Электронный ресурс. URL: https://developer.apple.com/documentation/foundation/userdefaults (дата обращения 29.04.26).
17. Apple Developer Documentation. Keychain Services. // Электронный ресурс. URL: https://developer.apple.com/documentation/security/keychain_services (дата обращения 29.04.26).
18. Apple Developer Documentation. SFSafariViewController. // Электронный ресурс. URL: https://developer.apple.com/documentation/safariservices/sfsafariviewcontroller (дата обращения 29.04.26).
19. Swift.org. Swift Testing. // Электронный ресурс. URL: https://www.swift.org/documentation/testing/ (дата обращения 29.04.26).
20. Point-Free. The Composable Architecture. // Электронный ресурс. URL: https://github.com/pointfreeco/swift-composable-architecture (дата обращения 29.04.26).
21. Исходный код проекта Sentinel iOS // GitHub репозиторий. – URL: https://github.com/cutesp1rit/Sentinel_coursework/tree/main/sentinel-ios (дата обращения 29.04.26).

ПРИЛОЖЕНИЕ 2. ТЕРМИНЫ И СОКРАЩЕНИЯ

1. API (Application Programming Interface) — программный интерфейс взаимодействия клиента с серверной частью по HTTPS и JSON.
2. AppSettings — локальная структура пользовательских настроек приложения, включающая Default Prompt, выбранное окружение и сведения о недавно активном чате.
3. Default Prompt — пользовательский текстовый шаблон, автоматически добавляемый к каждому сообщению в чате и к каждому запросу ребалансировки.
4. EventKit — системный фреймворк Apple для доступа к календарям и событиям устройства.
5. Flow-тест — автоматизированный тест, проверяющий последовательность действий, эффектов и переходов состояния внутри пользовательского сценария.
6. JSON (JavaScript Object Notation) — формат представления структурированных данных, используемый в сетевых контрактах приложения.
7. Keychain — защищённое системное хранилище Apple для токенов и иных чувствительных данных.
8. Rebalance — функция приложения Sentinel для получения и применения предложения по перераспределению событий на выбранном диапазоне дней.
9. Resource Battery — вычисляемая модель загруженности расписания пользователя на ближайший период или на отдельный день.
10. SFSafariViewController — системный компонент iOS для отображения веб-страниц внутри приложения.
11. Smoke-тест — быстрая проверка того, что экран или базовый сценарий может быть создан и отрисован без ошибки.
12. SwiftData — технология локального хранения данных, используемая в проекте для записей синхронизации календаря.
13. Swift Testing — фреймворк тестирования на Swift, применяемый в таргете SentinelTests.
14. SwiftUI — декларативный фреймворк Apple для построения пользовательского интерфейса.
15. TCA (The Composable Architecture) — архитектурный подход с однонаправленным потоком данных, лежащий в основе клиентского приложения Sentinel.
16. TestStore — инструмент TCA для тестирования редьюсеров, их эффектов и переходов состояний.
17. UserDefaults — системное хранилище настроек пользователя, используемое в проекте для AppSettings и service-флагов.
18. View — экран или визуальный компонент пользовательского интерфейса в SwiftUI.

19. Reducer — модуль бизнес-логики ТСА, преобразующий состояние приложения в ответ на действия пользователя и эффекты.

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц в докум.)	№ документа	Входящий № сопроводительного докум. и дата	Подп.	Дата
Изм.	Измененных	Замененных	Новых	Аннулированных					